# CinemaCore Systems

# Software Requirements Specification

# Version 2.4

# 9 April 2026

Group #9

Ryann Silverio, Mehar Buttar, William Gray

Prepared for
CS 250- Introduction to Software Systems
Instructor: Gus Hanna, Ph.D.
Spring 2026

## Revision History

| Date | Description | Author | Comments |
|---|---|---|---|
| 2/19/2026 | Version 1 | Group 9 | First Revision |
| 3/5/2026 | Version 1.7 | Group 9 | Second Revision |
| 3/19/2026 | Version 2.0 | Group 9 | Third Revision |
| 4/9/2026 | Version 2.4 | Group 9 | Fourth Revision |

## Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

| Signature | Printed Name | Title | Date |
|---|---|---|---|
| | Group 9 | Software Eng. | |
| | Dr. Gus Hanna | Instructor, CS 250 | |
| | | | |

## Github Link: https://github.com/ryannsilverio/cinemacore-systems-srs

# Table of Contents

**Revision History** 2
**Document Approval** 2
**Github Link: https://github.com/ryannsilverio/cinemacore-systems-srs** 2
**1. Introduction** 1
1.1 Purpose 1
1.2 Scope 1
1.3 Definitions, Acronyms, and Abbreviations 1
1.4 References 3
1.5 Overview 3
**2. General Description** 4
2.1 Product Perspective 4
2.2 Product Functions 4
2.3 User Characteristics 5
2.4 General Constraints 5
2.5 Assumptions and Dependencies 5
**3. Specific Requirements** 6
3.1 External Interface Requirements 6
*3.1.1 User Interfaces* 6
*3.1.2 Hardware Interfaces* 7
*3.1.3 Software Interfaces* 7
*3.1.4 Communications Interfaces* 7
3.2 Functional Requirements 7
*3.2.1 Functional Requirement #1: Movie Selection and Interactive Seat Reservation* 8
*3.2.2 Functional Requirement #2: Ticket Purchase, Delivery, and Validation* 8
*3.2.3 Functional Requirement #3: Virtual Concession Ordering and Seat Delivery* 9
3.3 Use Cases 10
*3.3.1 Use Case #1: Browse Movies and Reserve Seats* 10
*3.3.2 Use Case #2: Purchase and Receive Movie Tickets* 11
*3.3.3 Use Case #3: Order Concessions for Seat Delivery* 11
*3.3.4 Use Case Diagram* 13
3.4 CinemaCore System Classes / Objects 14
*3.4.1 CinemaCore System Classes UML Diagram* 14
*3.4.2 Classes Descriptions* 15
3.5 Non-Functional Requirements 18
*3.5.1 Performance:* 18
*3.5.2 Reliability:* 18
*3.5.4 Security:* 19
*3.5.5 Maintainability* 19
*3.5.6 Portability* 19
3.6 Inverse Requirements 19
*3.6.1 Seat and Booking Constraints* 19
*3.6.2 Payment and Security Constraints* 20
*3.6.3 Age-Restricted Content Constraints* 20
*3.6.4 Concession Ordering Constraints* 20
*3.6.5 Ticket Delivery Constraints* 20

3.6.6 Performance and Access Constraints....20
**4. System Overview....21**
4.1 Developer System Perspective....21
4.2 Architectural Design....22
*4.2.1 Architectural Diagram....22*
*4.2.2 Architectural Description....23*
4.3 Software Architecture (SWA) Design....24
*4.3.1 SWA Diagram....24*
*4.3.2 SWA Description....25*
*4.3.3 SWA Revisions....26*
4.4 Development Plan....26
*4.4.1 Task Breakdown....26*
*4.4.2 Project Timeline....28*
Project Timeline....28
4.5 Test Plan....29
*4.5.1 Test Scope and Coverage Strategy....29*
*4.5.2 Test Priority....29*
*4.5.3 Test Case Specification....29*
*4.5.4 Test Plan Table....34*
**5. Data Management Strategy....35**
5.1 Technology Choice....35
5.2 Database Organization....36
*5.2.1 Users....36*
*5.2.2 Movies & Showtimes....37*
*5.2.3 Theaters & Seats....37*
*5.2.4 Tickets & Orders....37*
5.3 Alternatives & Tradeoffs....38
5.4 Security Considerations....39

# 1. Introduction

The Software Requirements Specification (SRS) introduction presents a comprehensive overview of the document structure, including its purpose, scope, terminology definitions, referenced materials, and organizational framework. This document aims to provide thorough analysis and detailed understanding of the CinemaCore Systems by clearly articulating the system objectives and requirements. Additionally, it focuses on identifying stakeholder capabilities and expectations while establishing the primary product features. Complete and specific requirements for the CinemaCore Systems are documented throughout this specification.

## 1.1 Purpose

This document serves to gather and evaluate all concepts and requirements that have emerged during system planning, with particular focus on customer needs and expectations. The document aims to clarify intended product usage patterns, enabling deeper project comprehension while documenting both potential features for future development and preliminary ideas that may be refined or eliminated as the system evolves.

Essentially, this SRS document delivers comprehensive details about the software product, including its specifications, objectives, and operational parameters. The document outlines the intended user base, interface design, and technical requirements for both hardware and software components. It establishes a unified vision of the product and its capabilities from the perspectives of clients, development teams, and end users. Moreover, it serves as an essential resource for designers and developers throughout all phases of the software development lifecycle (SDLC).

## 1.2 Scope

The scope encompasses the CinemaCore System's functionality for deploying a complete theater ticketing and service platform. It addresses theater management needs, stakeholder expectations, and system features that support ticket sales, dynamic pricing, seat selection, virtual concession ordering, and customer interaction across both digital and physical channels.

This SRS aims to define requirements for the software under development, though it may additionally assist in selecting appropriate commercial or in-house software solutions. The specification can be utilized to generate software requirements directly or serve as a template for establishing project-specific or organizational standards. It does not mandate specific methods, naming conventions, or development tools for creating an SRS.

## 1.3 Definitions, Acronyms, and Abbreviations

| **Term** | **Definition** |
|---|---|
| **SRS** | Software Requirements Specification |

| | |
|---|---|
| **POS** | Point of Sale; the system interface where transactions are completed |
| **API** | Application Programming Interface; protocols for software communication |
| **SMTP** | Simple Mail Transfer Protocol |
| **SMS** | Short Message Service; text messaging |
| **PCI-DSS** | Payment Card Industry Data Security Standard |
| **ADA** | Americans with Disabilities Act; accessibility compliance standards |
| **MPAA** | Motion Picture Association of America; organization that assigns movie ratings |
| **HTTP/HTTPS** | Hypertext Transfer Protocol (Secure); protocol used for transmitting web pages and data over the internet with encryption |
| **TCP/IP** | Transmission Control Protocol/Internet Protocol; the basic communication language or protocol of the internet |
| **Adyen** | Third-party payment processing platform that securely handles online and in-person transactions in compliance with PCI-DSS standards |
| **WebSocket** | A communication protocol that enables a persistent, two-way connection between a client (like a web browser) and a server. |
| **DB** | Database |
| **SQL** | A language used to store, retrieve, and manage data in relational databases. |
| **PostgreSQL** | An open-source relational database system that uses SQL and is known for reliability and advanced features. |
| **JWT** | JSON Web Token, a compact, secure way to transmit information (like user identity) between parties as a signed token, often used for authentication. |
| **ACID** | A set of properties that ensure reliable database transactions:<br>● **A**tomicity (all or nothing)<br>● **C**onsistency (valid state)<br>● **I**solation (transactions don't interfere)<br>● **D**urability (data persists after commit) |
| **Redis** | A fast, in-memory database often used for caching and real-time data. |

| | |
|---|---|
| **PK** | Primary Key |
| **Bcrypt** | A secure algorithm used to hash (protect) passwords. |
| **Native** | Built-in or supported directly by a system without needing extra tools or layers. |
| **Native FK** | Native Foreign Key, a built-in database constraint that links one table to another using a foreign key. |
| **MongoDB** | A NoSQL database that stores data in flexible, JSON-like documents instead of tables. |
| **TLS** | Transport Layer Security, a protocol that encrypts data sent over the internet to keep it secure. |

## 1.4 References

Adyen. (n.d.). *Adyen payments platform*. https://www.adyen.com

Apple Inc. (n.d.). *Wallet developer documentation*. Apple Developer. https://developer.apple.com/wallet/

Google. (n.d.). *Google Pay API documentation*. Google Developers. https://developers.google.com/pay

IEEE Computer Society. (1998). *IEEE recommended practice for software requirements specifications (IEEE Std 830-1998)*. Institute of Electrical and Electronics Engineers.

Motion Picture Association. (n.d.). *Film ratings*. https://www.motionpictures.org/film-ratings/

PCI Security Standards Council. (n.d.). *Payment Card Industry (PCI) data security standard (PCI DSS)*. https://www.pcisecuritystandards.org/

World Wide Web Consortium. (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. https://www.w3.org/TR/WCAG21/

## 1.5 Overview

The subsequent sections of this document deliver a comprehensive description, encompassing user characteristics, hardware specifications, and both functional and data requirements for the system. Section 2 discusses the general project description and overall system perspective. Section 3 presents functional requirements, data specifications, design constraints, and assumptions established during CinemaCore System development. This section also captures the user perspective of the product and provides detailed specific requirements including external interface specifications and comprehensive functional requirement descriptions. Section 4 contains analysis models with sequence diagrams, data flow diagrams, and state-transition

diagrams for requirement visualization. Section 5 describes the change management process for handling requirement modifications throughout the project lifecycle.

# 2. General Description

This section presents the problem statement addressing current limitations in traditional movie theater ticketing operations that restrict customer convenience and operational efficiency. It includes identification of stakeholders and end users for the proposed CinemaCore Systems. Additionally, it outlines stakeholder needs and expectations gathered through requirements workshops and analysis sessions. The section lists and describes the primary features and capabilities of the system.

The SRS provides a comprehensive product perspective from various stakeholder viewpoints. It details the product functions of CinemaCore System including user characteristics, design constraints, system assumptions and dependencies, and requirement classifications.

## 2.1 Product Perspective

CinemaCore System is a comprehensive point-of-sale platform that can be implemented as a standalone solution for new theaters or integrated as an upgrade to existing theater management systems. The system is designed with flexibility to adapt to various theater infrastructures, allowing seamless integration with legacy ticketing systems, scheduling software, and operational databases.

The system interfaces with external payment gateways (Visa, Mastercard, PayPal), digital wallet services (Apple Wallet, Google Pay), email and SMS providers for notifications, tax calculation services, and age verification databases. For theaters with existing food service operations, the system integrates with current point-of-sale systems to coordinate virtual orders with kitchen and delivery operations.

The system supports standard theater hardware including kiosk terminals, receipt printers, barcode scanners, and display monitors. For theaters upgrading from legacy systems, data migration capabilities preserve historical customer information, transaction records, and operational data during platform transition.

## 2.2 Product Functions

The functions performed by the system are:

- Ticket purchasing and payment processing with discount and surcharge application
- Real-time seat selection and showroom visualization
- Dynamic pricing with tiered seating options (cheaper front sections, premium reserved seating)
- Virtual food ordering with seat delivery coordination
- Customer account creation and profile management
- Email receipt generation and delivery
- Multi-format ticket access (email, SMS, print, digital wallet)

- Timed checkout sessions to ensure fair access during high-demand periods
- Virtual waiting room for managing concurrent ticket purchases
- Age verification for R-rated movies and alcohol purchases
- Ticket counter displaying remaining seat availability per showing
- Showtime reminder notifications
- Ticket transfer functionality for families and resellers
- Physical kiosk support with integrated printer capabilities
- Loyalty points and rewards program
- Movie information management (ratings, showtimes, descriptions)
- Administrative interfaces for theater staff and system management

## 2.3 User Characteristics

Different user types include:

- **Children**: Require simple, intuitive interfaces with parental guidance; limited purchasing power
- **Teenagers**: Tech-savvy, prefer mobile/digital wallets; may need age verification for certain content
- **Adults**: Primary purchasers; varied technical proficiency; value convenience and speed
- **Elderly patrons**: May prefer physical tickets and kiosk options; need clear, accessible interfaces with larger text
- **Theater staff**: Need administrative access for order management, seat checking, food delivery coordination
- **System administrators**: Technical users managing system configuration, pricing, and maintenance

## 2.4 General Constraints

Limitations that may affect the development of CinemaCore Systems include:

- **Regulatory constraints**: PCI-DSS compliance for payment processing; age verification laws for R-rated content and alcohol
- **Hardware limitations**: Must run on standard theater kiosk hardware with receipt printers
- **Internet connectivity**: System requires reliable internet for real-time seat updates and payment processing
- **Security requirements**: Must protect customer payment information and personal data
- **Accessibility standards**: Must comply with ADA requirements for disabled users
- **Browser compatibility**: Must support major web browsers (Chrome, Firefox, Safari, Edge)
- **Mobile wallet integration**: Limited to APIs provided by Apple Wallet and compatible platforms

## 2.5 Assumptions and Dependencies

**Assumptions:**

- Users have access to email addresses and phone numbers
- Theater has reliable internet connectivity
- Printers at kiosks are maintained and operational
- Credit card payment gateways are available and functional
- Users have basic computer/smartphone literacy
- Theater staff available to deliver food orders to seats

**Dependencies:**

- Third-party payment gateway availability (Stripe, PayPal, etc.)
- SMTP server functionality
- Digital wallet API availability (Apple Wallet, Google Pay)
- Age verification service database access
- Tax calculation service availability
- Printer driver compatibility with kiosk hardware
- MPAA ratings accessibility

# 3. Specific Requirements

## 3.1 External Interface Requirements

### 3.1.1 User Interfaces

The system will provide three distinct user interface categories designed for high accessibility and usability across all age groups.

- **Customer Web Interface**: A responsive web browser application.
    - **Dashboard**: A full-service view showing movies, ratings, showtimes, and real-time seat availability.
    - **Seat Selection Map**: A graphical theater preview identifying taken vs. available seats, with color-coded pricing tiers (i.e., discounted front row vs. premium center section)
    - **Checkout Flow**: A streamlined, timed checkout process with a visible countdown timer.
    - **Virtual Concession Stand**: An intuitive menu for food and drink selection with a toggle option for "deliver to my seat."
- **Physical Kiosk Interface**: A simplified, touch-optimized version of the web interface made for use at the movie theater.
    - **Interaction**: Large buttons and text for clear and easy navigation.
    - **Navigation**: Linear flow (Select Movie → Select Time → Select Seats → Payment → Print/Digital Delivery).
- **Theater Staff Interface**: A management dashboard for real-time operations by theater staff.
    - **Food Service View**: A management system showing order codes corresponding to theater rooms and seat numbers.

- **Entry Management**: A scanning interface for ticket validation and age verification (R-rated movies/alcohol)

### 3.1.2 Hardware Interfaces

The system will interface with the following physical hardware components:

- **Kiosk Terminals**: Support for standard touch-screen POS hardware located in theater lobbies.
- **Receipt Printers**: Standard printers to produce physical tickets and food receipts for guests without digital receipts.
- **Barcode/QR Code Scanners**: Integration with handheld and fixed scanners to validate virtual and physical tickets.
- **Payment Terminals**: Connection to physical card readers for tap payment and chip insertions at kiosks.

### 3.1.3 Software Interfaces

The system will exchange data with the following external software entities:

- **Payment Gateways**: Secure API connections for credit and debit card processing.
- **Digital Wallet APIs**: Integration with Apple Wallet and Google Pay for pushing virtual tickets to user devices.
- **Notification Services**: For sending emailed receipts and account verification links.
    - **SMS Gateway**: For sending showtime reminders and phone-based ticket links.
- **Age Verification Service**: API to databases or ID scanning software to validate age for R-rated movies and alcohol sales.
- **Tax Calculation API**: Connection to a third-party service for real-time localized tax application on tickets and concessions.

### 3.1.4 Communications Interfaces

To ensure data integrity and security, the system will utilize the following communication protocols:

- **HTTP/HTTPS**: All web traffic shall be encrypted using standard HTTP/HTTPS.
- **TCP/IP**: For local communication between kiosk hardware, printers, and the theater's central server.
- **Real-Time Synchronization**: Use of WebSockets or long-polling for updating seat availability and virtual waiting room status for all concurrent users.

## 3.2 Functional Requirements

This section defines the core functions and behaviors that CinemaCore Systems must perform to meet user and stakeholder needs. Each functional requirement specifies system inputs, processing logic, outputs, and error handling associated with a particular feature. These requirements describe *what* the system must do and serve as the primary basis for use case development, system design, and requirement validation.

### 3.2.1 Functional Requirement #1: Movie Selection and Interactive Seat Reservation

**3.2.1.1 Introduction**: This feature allows the user to browse movies, view showtimes, and select specific seats from a virtual representation of the movie theater. It incorporates dynamic pricing based on seat location and manages a timed checkout.

**3.2.1.2 Inputs**: User selection for specific movie, date, and time.

- **Seat Locations**: User selected seats from the graphical theater map.
- **User Age**: Tickets categorized by age for initial pricing.
- **Checkout Timer**: Reservation timer activated on first seat selection.

**3.2.1.3 Processing**: Real-time seat availability check in the database.

- **Tiered Pricing**: The system will apply a surcharge for premium sections and will apply a discount for front row seating.
- **Seat Locking**: Upon selection, the system will temporarily lock the seats from other customers for 10 minutes. If the seats are purchased, the system will permanently lock the seats unless refunded.
- **User Traffic Management**: If high traffic is detected, the system will place subsequent users in a virtual waiting room queue before allowing seat selection.

**3.2.1.4 Outputs**: Real-time updates of theater maps showing selected seats as "Pending Purchase."

- **Price Summary**: A breakdown of base price, location-based surcharges and discounts, and age-based discounts.
- **Countdown Timer**: A visible 10 minute timer displayed for the user to see.

**3.2.1.5 Error Handling**: If the 10 minute timer hits zero, the system will release the locked seats, clear the user's cart, and notify the user via a pop-up message.

### 3.2.2 Functional Requirement #2: Ticket Purchase, Delivery, and Validation

**3.2.2.1 Introduction**: This feature allows customers to securely purchase movie tickets and receive them in multiple formats, including digital and physical options. Tickets must be uniquely identified and verifiable at theater entry points.

**3.2.2.2 Inputs**: Confirmed user selected seats and user information

- **Selected seats:** Confirmed selected seats
- **Payment information:** Credit/debit card or digital wallet
- **Form of contact:** Email address and/or phone number
- **Option to save info:** Optional login credentials with email and password

**3.2.2.3 Processing**: Real-time payment and account validation

- **Secure payment**: The system shall securely process payments in compliance to PCI-DSS standards

- **Unique Ticket Identifier**: Upon successful payment, the system will generate a unique ticket identification number and associated barcode
- **Ticket Linkage**: The system shall link purchased tickets to the associated screening room, showtime, and seat number

**3.2.2.4 Outputs**: Summary of purchases and options for digital or physical forms

- **Purchase Receipt**: Confirmation receipt detailing purchase information, cost, payment used, and seats
- **Digital Ticket**: A digital ticket sent via email, SMS, and/or digital wallet
- **Printed Ticket:** Optional printed ticket generated at physical kiosk locations

**3.2.2.5 Error Handling**:

- If the payment authorization fails, the system shall cancel the transaction, release the locked seats, and display an appropriate error message
- If ticket delivery fails, the system shall retry delivery and provide the user with an on-screen recovery option

### 3.2.3 Functional Requirement #3: Virtual Concession Ordering and Seat Delivery

**3.2.3.1 Introduction**: This feature allows customers to browse, purchase, and manage concession items digitally as part of the movie ticketing experience. This feature enables customers to order food and beverages in advance or during the movie, and have items delivered directly to their seats

**3.2.3.2 Inputs**: Food items, digital movie ticket identifier, and user payment info

- **Concession:** Selected food and beverage items, as well as quantity of each
- **Ticket:** Contains seat location and screening room
- **Confirmation:** Status that payment has been confirmed

**3.2.3.3 Processing**: Real-time payment and account validation, and food availability and cost

- **Menu:** The system shall display real-time concession items on a menu, with current pricing and availability
- **Linkage to Ticket Identifier**: Upon successful payment, the system will link the concession items to the user's ticket identifier
- **Food Transmission**: The system shall transmit food orders to the theater staff interface with order status indicators (e.g. Received, In Preparation, Out for Delivery)
- **Age Verification:** The system shall enforce age verification for alcoholic beverages in compliance with local regulations
- **Cutoff Time:** The system shall prevent concession orders once a predefined cutoff time during the movie has been reached

**3.2.3.4 Outputs**: Summary of purchases with estimated delivery time, as well as order status

- **Purchase Receipt**: Confirmation receipt detailing purchase information, cost, payment used, and seats, and delivery estimated time of arrival
- **Digital Receipt**: A digital receipt sent via email and a summary of orders available through user interface
- **Order Status**: Real-time status updates available through the customer interface
- **Printed Receipt:** Optional printed receipt delivered to customer with food, option to print is available through the customer interface

**3.2.3.5 Error Handling**:

- If a selected item becomes unavailable, the system shall notify the user and prompt for a replacement or removal
- If order transmission fails, the system shall retry delivery and alert theater staff if manual intervention is required

## 3.3 Use Cases

This section outlines the primary use cases of the system, describing how users and external entities interact with the system to achieve specific goals. Each use case defines the sequence of actions between the actor and the system, providing a functional view of the system's behavior and serving as a basis for validating requirements and guiding system design and implementation.

### 3.3.1 Use Case #1: Browse Movies and Reserve Seats

This use case describes how a customer selects a movie and showtime, views real-time seat availability, and reserves seats using an interactive theater layout with dynamic pricing and a timed checkout process.

**3.3.1.1 Associated Requirement:**

- FR-1: Movie Selection and Interactive Seat Reservation

**3.3.1.2 Primary Actor:**

- Adult customer

**3.3.1.3 Basic Flow:**

1. The customer accesses CinemaCore Systems via web or kiosk
2. The customer browses available movies and selects a showtime
3. The system displays a graphical seat map with real-time availability
4. The customer selects one or more seats
5. The system applies tiered pricing based on seat location
6. The system locks the selected seats and starts the 10-minute checkout timer

**3.3.1.4 Alternative Flow:**

- If the system traffic is high, the system will place the customer in a virtual waiting room until access becomes available

**3.3.1.5 Preconditions:**

- The system is online and operational
- Movies and Showtimes are available

**3.3.1.6 Postconditions:**

- Selected seats are temporarily locked for the customer

- A checkout timer is activated

**3.3.1.7 Special Requirements:**

- Seat availability must update in real-time across all users
- The interface must comply with ADA accessibility requirements
- Seat-locking must prevent double-booking under concurrent access

### 3.3.2 Use Case #2: Purchase and Receive Movie Tickets

This use case outlines the process by which a customer completes payment for reserved seats, receives tickets in digital or physical formats, and obtains a confirmation receipt for theater entry.

**3.3.2.1 Associated Requirement:**

- FR-2: Ticket Purchase, Delivery, and Validation

**3.3.2.2 Primary Actor:**

- Adult customer

**3.3.2.3 Basic Flow:**

1. The customer enters payment and contact information
2. The system securely processes the payment
3. The system generates a unique ticket identifier and barcode
4. The system delivers tickets via email, SMS, digital wallet, or kiosk printer
5. The system sends a purchase confirmation receipt

**3.3.2.4 Alternative Flow:**

- The customer selects physical ticket printing at a theater kiosk

**3.3.2.5 Preconditions:**

- Seats have been selected and locked
- Customer has a valid payment method

**3.3.2.6 Postconditions:**

- Tickets are generated and delivered in selected format
- Seats are permanently reserved for the showtime

**3.3.2.7 Other Stakeholders:**

- Payment processors
- Theater entry staff
- Compliance auditors

**3.3.2.8 System/subsystem:**

- Payment processing subsystem
- Ticket generation subsystem
- Notification and delivery subsystem
- Ticket validation subsystem

**3.3.2.9 Special Requirements:**

- Payment processing must comply with PCI-DSS standards
- Tickets must be uniquely identified and non-duplicate
- Digital tickets must be scannable both online and offline at theater entry points

### 3.3.3 Use Case #3: Order Concessions for Seat Delivery

This use case details how a customer orders food and beverages through the system, links the order to a specific seat and ticket, and receives in-seat delivery coordinated by theater staff.

**3.3.2.1 Associated Requirement:**

- FR-3: Virtual Concession Ordering and Seat Delivery

**3.3.2.2 Primary Actor:**

- Adult customer

**3.3.2.3 Basic Flow:**

1. The customer browses the virtual concession menu
2. The customer selects food and beverage items
3. The system links the order to the customer's ticket and seat
4. The customer confirms the order
5. The system sends the order to the theater staff interface
6. Theater staff prepares and delivers the order

**3.3.2.4 Alternative Flow:**

- The customer schedules the order for delivery before the movie starts

**3.3.2.5 Preconditions:**

- Customer has a valid ticket for showtime
- Concession services are enabled for the theater

**3.3.2.6 Postconditions:**

- Food is delivered to the correct seat
- Order status is marked as completed

**3.3.2.7 Other Stakeholders:**

- Theater Operations Management
- Health and Safety Inspectors

**3.3.2.8 System/subsystem:**

- Virtual Concession subsystem
- Order management subsystem
- Staff operations interface

**3.3.2.9 Special Requirements:**

- Alcohol purchases require age verification
- Orders must include clear seat and theater identifiers for delivery accuracy
- Order status updates must be visible to both customers and staff in real-time

### 3.3.4 Use Case Diagram

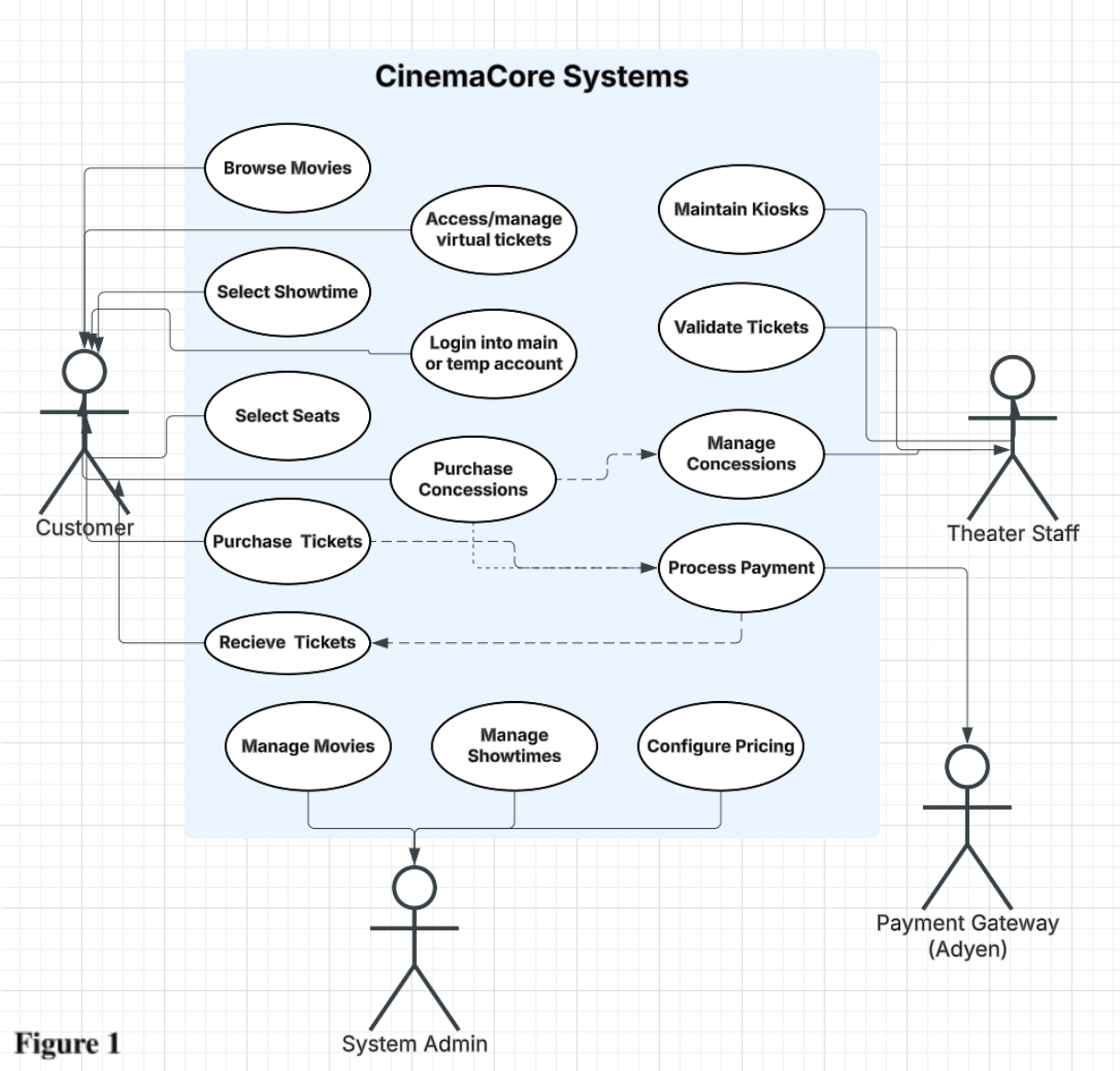


**Figure 1**

## 3.4 CinemaCore System Classes / Objects

### 3.4.1 CinemaCore System Classes UML Diagram

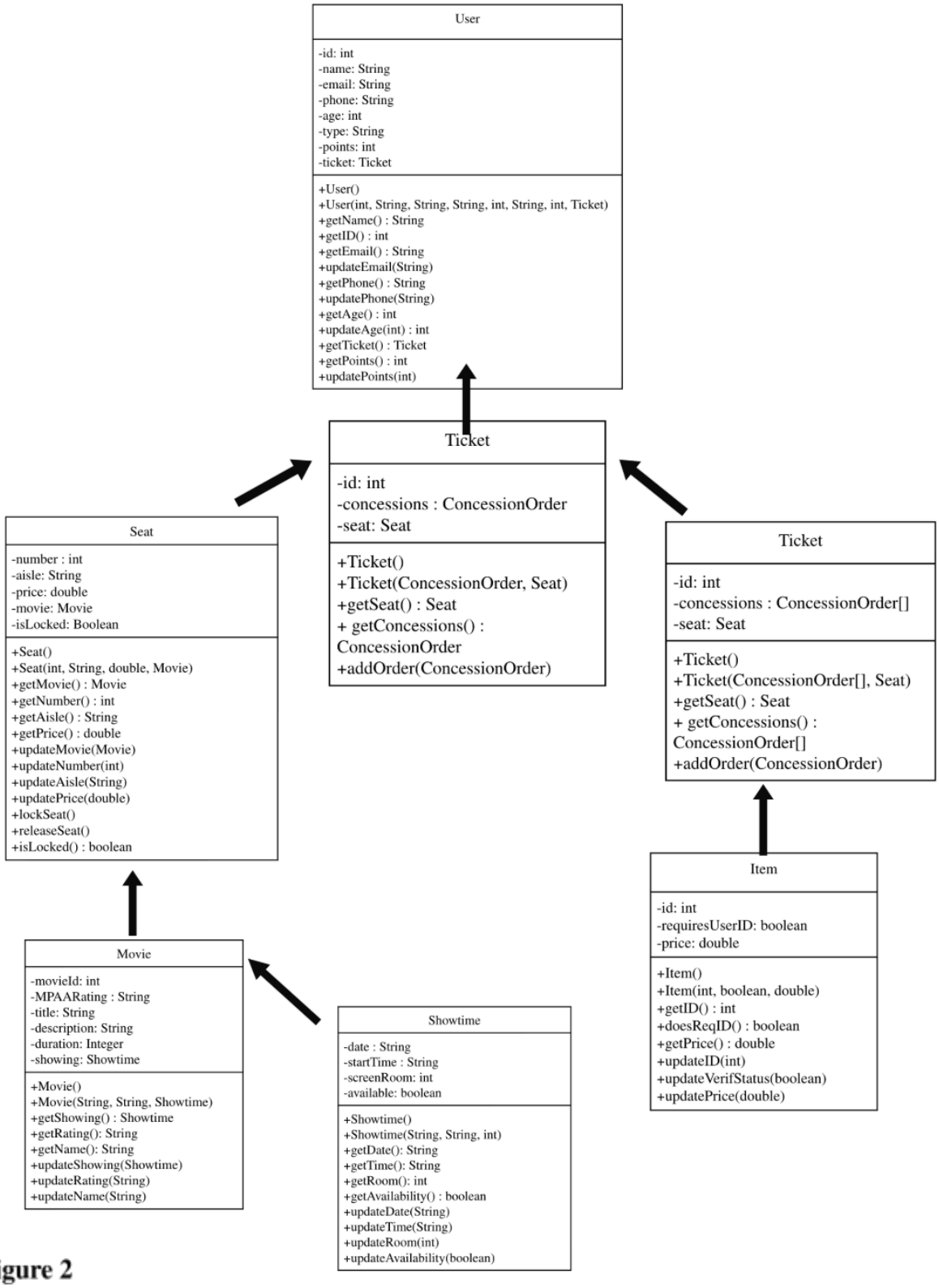


**Figure 2**

### 3.4.2 Classes Descriptions

#### 3.4.2.1 User

The **User** class represents an individual customer within the cinema system. It stores identifying and contact information as well as account-related data and the ticket associated with the user's purchase.

**Attributes:**

- id : int - Unique identifier for the user.
- name : String - Full name of the user.
- email : String - Email address used for communication and account identification.
- phone : String - Contact phone number.
- age : int - Age of the user, used for age-restricted item verification.
- type : String - Account or membership type (e.g., standard, rewards member).
- points : int - Reward points accumulated by the user.
- ticket : Ticket - Reference to the ticket associated with the user's purchase.

**Operations:**

- User() - Default constructor.
- User(int id, String name, String email, String phone, int age, String type, int points, Ticket ticket) - Parameterized constructor.
- String getName() - Returns the user's name.
- int getID() - Returns the user's unique ID.
- String getEmail() - Returns the user's email address.
- void updateEmail(String email) - Updates the user's email address.
- String getPhone() - Returns the user's phone number.
- void updatePhone(String phone) - Updates the user's phone number.
- int getAge() - Returns the user's age.
- void updateAge(int age) - Updates the user's age.
- Ticket getTicket() - Returns the associated ticket.
- int getPoints() - Returns the user's reward points.
- void updatePoints(int points) - Updates the user's reward point total.

#### 3.4.2.2 Ticket

The **Ticket** class represents a transaction connecting a user to a reserved seat and any associated concession orders. It acts as the central object linking seating and optional purchases.

**Attributes:**

- id : int - Unique identifier for the ticket.
- concessions : ConcessionOrder[] - Concession order associated with the ticket.
- seat : Seat - The seat reserved by the ticket.

**Operations:**

- Ticket() - Default constructor.
- Ticket(ConcessionOrder[] concessions, Seat seat) - Parameterized constructor.
- Seat getSeat() - Returns the seat associated with the ticket.
- ConcessionOrder[] getConcessions() – Returns the concession order associated with the ticket.

- void addOrder(ConcessionOrder order) - Attaches a concession order to the ticket.

**3.4.2.3 Seat**

The **Seat** class represents a physical seat within a theater for a particular movie showing. It stores seat location, price, associated movie, and reservation status.

**Attributes:**

- number : int - Seat number within the row.
- aisle : String - Aisle or section identifier.
- price : double - Price of the seat.
- movie : Movie - Movie currently associated with the seat.
- isLocked : Boolean - Indicates whether the seat is currently reserved/locked.

**Operations:**

- Seat() - Default constructor.
- Seat(int number, String aisle, double price, Movie movie) - Parameterized constructor.
- Movie getMovie() - Returns the associated movie.
- int getNumber() - Returns the seat number.
- String getAisle() - Returns the aisle identifier.
- double getPrice() - Returns the seat price.
- void updateMovie(Movie movie) - Updates the movie assigned to the seat.
- void updateNumber(int number) - Updates the seat number.
- void updateAisle(String aisle) - Updates the aisle identifier.
- void updatePrice(double price) - Updates the seat price.
- void lockSeat() - Locks the seat to prevent other reservations.
- void releaseSeat() - Releases the seat and makes it available again.
- boolean isLocked() - Returns whether the seat is currently locked.

**3.4.2.4 Movie**

The **Movie** class represents a film available in the cinema system. It stores identifying information about the film and its associated showtime.

**Attributes:**

- movieID : int - Unique identifier for the movie.
- MPAARating : String - MPAA rating (e.g., G, PG, PG-13, R).
- title : String - Name of the movie.
- description : String - Brief summary of the movie.
- duration : Integer - Length of the movie in minutes.
- showing : Showtime - Showtime associated with the movie.

**Operations:**

- Movie() - Default constructor.
- Movie(String title, String rating, Showtime showing) - Parameterized constructor.
- Showtime getShowing() - Returns the showtime associated with the movie.
- String getRating() - Returns the movie's MPAA rating.
- String getName() - Returns the movie title.
- void updateShowing(Showtime showing) - Updates the showtime.
- void updateRating(String rating) - Updates the movie's rating.

- void updateName(String name) - Updates the movie title.

**3.4.2.5 Showtime**

The **Showtime** class represents a scheduled screening for a movie. It contains information about when and where the showing occurs and whether tickets are still available.

**Attributes:**

- date : String - Date of the screening.
- startTime : String - Start time of the movie.
- screenRoom : int - Theater room number where the movie will play.
- available : boolean - Indicates whether the showing is still open for reservations.

**Operations:**

- Showtime() - Default constructor.
- Showtime(String date, String startTime, int screenRoom) - Parameterized constructor.
- String getDate() - Returns the date of the showing.
- String getTime() - Returns the start time.
- int getRoom() - Returns the screen room number.
- boolean getAvailability() - Returns whether the showing is available.
- void updateDate(String date) - Updates the show date.
- void updateTime(String time) - Updates the start time.
- void updateRoom(int room) - Updates the theater room number.
- void updateAvailability(boolean available) - Updates the availability status.

**3.4.2.6 ConcessionOrder**

The **ConcessionOrder** class represents a collection of concession items purchased alongside a ticket. It manages a list of items and provides functionality for calculating the total cost.

**Attributes:**

- items : Item[] - Array containing all items included in the order.
- id : int - Unique identifier for the concession order.

**Operations:**

- ConcessionOrder() - Default constructor.
- ConcessionOrder(Item[] items, int id) - Parameterized constructor.
- Item[] getItems() - Returns the list of items in the order.
- void addItem(Item item) - Adds a new item to the order.
- int getID() - Returns the order ID.
- double calculatePrice() - Calculates and returns the total price of all items in the order.

**3.4.2.7 Item**

The **Item** class represents an individual concession product that may be included in a concession order.

**Attributes:**

- id : int - Unique identifier for the item.

- requiresUserID : boolean - Indicates whether ID verification is required (e.g., alcohol purchases).
- price : double - Price of the item.

**Operations:**

- Item() - Default constructor.
- Item(int id, boolean requiresUserID, double price) - Parameterized constructor. int getID() - Returns the item ID.
- boolean doesReqID() - Returns whether the item requires ID verification.
- double getPrice() - Returns the price of the item.
- void updateID(int id) - Updates the item ID.
- void updateVerifStatus(boolean requiresUserID) - Updates the ID verification requirement.
- void updatePrice(double price) - Updates the item price.

## 3.5 Non-Functional Requirements

This section specifies the quality attributes and operational constraints that CinemaCore Systems must satisfy. Non-functional requirements address system-wide characteristics such as performance, reliability, availability, security, maintainability, and portability. These requirements define *how well* the system must operate and establish measurable criteria used to evaluate system effectiveness and compliance.

### 3.5.1 Performance:

- The system should not have noticeable delays when transferring ticket selection data or updating seat availability. Seat availability must update in real time across all users so that double-booking does not occur.
- The system requires a constant connection to the main server that stores ticketing information. A stable network connection is necessary for real-time seat updates and payment processing. To reduce latency, the system should be optimized to minimize unnecessary network traffic between kiosks and the central server.
- At least 95% of checkout transactions (from payment submission to confirmation) should be processed in less than one second, excluding delays caused by external payment providers such as Adyen.

### 3.5.2 Reliability:

- The software must reliably prevent double-booking under all conditions. Once a seat is locked or purchased, it must not be available to other users unless the hold expires or a refund is processed.
- If a network issue occurs, the system may operate in a limited offline mode at kiosk locations. In this mode, tickets may be printed physically on-site, but digital delivery features may be unavailable until the network connection is restored.
- The system should maintain an uptime target of 99.9% during theater operating hours, excluding scheduled maintenance.

### 3.5.3 Availability:

- The system must remain accessible during all standard theater operating hours.

- Multiple kiosks at each location should ensure that ticket purchasing remains available even if one kiosk experiences a hardware issue.
- Planned maintenance should be scheduled during non-peak hours to minimize disruption to customers.

### 3.5.4 Security:

- The system must protect customer data and payment information at all times.
- All web traffic must be encrypted using HTTPS.
- Payment processing will be outsourced through Adyen in compliance with PCI-DSS standards. The system will not store full credit card information locally.
- Administrative access will require secure login credentials, and only authorized staff will be able to modify showtimes, pricing, and availability.
- Tickets purchased without a registered account can be accessed using the email address provided at checkout and a temporary access code sent to that email.
- CAPTCHA and queue-based traffic control mechanisms will be used to prevent bot activity during high-demand events.

### 3.5.5 Maintainability

- The system should be designed so that pricing, showtimes, and seat availability can be updated through the admin interface
- Basic hardware issues should be diagnosable by theater staff, with maintenance techs available when necessary.

### 3.5.6 Portability

- The customer interface must function across major web browsers (Chrome, Firefox, Safari, Edge).
- The kiosk version must be compatible with standard touchscreen POS hardware used in theaters.
- After purchase, tickets must be easily accessible on mobile devices via email, SMS link, or digital wallet, even if originally purchased at a kiosk.

## 3.6 Inverse Requirements

This section defines behaviors and actions that the CinemaCore System must explicitly never perform. These requirements serve as constraints that bound acceptable system behavior and are derived directly from the functional and non-functional requirements defined in this document.

### 3.6.1 Seat and Booking Constraints

The system shall not:

- Allow a seat to be selected or purchased by another user while it is temporarily locked or permanently reserved.
- Permanently lock a seat if payment was not completed or the 10-minute checkout timer has expired.
- Generate duplicate tickets for the same seat and showtime under any circumstances.
- Leave seats in a “held” state indefinitely; all holds must expire or convert to a completed purchase.

### 3.6.2 Payment and Security Constraints

The system shall not:

- Store full credit card numbers or sensitive cardholder data locally on any kiosk terminal or central server.
- Process any payment transaction outside of PCI-DSS compliant encryption standards.
- Grant administrative access to pricing, showtimes, or seat availability without verified staff credentials.
- Display or expose sensitive payment details (full card number, CVV) in receipts, logs, or error messages.

### 3.6.3 Age-Restricted Content Constraints

The system shall not:

- Permit the purchase of R-rated movie tickets or alcoholic concession items without successful age verification.
- Bypass age verification requirements regardless of user account type, loyalty status, or purchase history.
- Approve an alcohol delivery order unless age verification is confirmed

### 3.6.4 Concession Ordering Constraints

The system shall not:

- Accept new concession orders after the predefined cutoff time during an active screening.
- Link a concession order to a seat that does not have a valid, confirmed ticket associated with it.
- Accept concession orders for items marked unavailable/out of stock at the time of checkout.
- Deliver concession orders without including clear auditorium/screening room and seat identifiers.

### 3.6.5 Ticket Delivery Constraints

The system shall not:

- Mark a ticket as successfully delivered if the email, SMS, or digital wallet transmission has failed without a confirmed successful retry.
- Allow ticket retrieval using email-only access without the temporary access code or equivalent verification step

### 3.6.6 Performance and Access Constraints

The system shall not:

- Permit unchecked automated bot activity during high-demand ticket release periods.
- Allow modifications to showtimes, pricing, or seat availability without administrator authentication.
- Allow a single user/session to hold an excessive number of seats beyond a defined limit
- Allow ticket validation scans to allow the same ticket more than once for entry

# 4. System Overview

This section presents the CinemaCore System from a developer perspective, describing the system's internal structure, component relationships, and technical implementation strategy. It provides the architectural foundation necessary for development teams to understand how system components interact and how the platform is organized for implementation and maintenance.

## 4.1 Developer System Perspective

CinemaCore Systems is a multi-tiered, service-oriented platform built to support real-time ticketing, concession ordering, and theater management operations. From a development standpoint, the system is composed of three primary layers: a presentation layer handling all customer and staff-facing interfaces, a business logic layer managing core processing such as seat locking, payment handling, and order routing, and a data layer responsible for persistent storage of user accounts, transactions, seat availability, and concession inventory.

The system is designed to support concurrent users across both web and kiosk interfaces, requiring thread-safe seat locking mechanisms and real-time database synchronization. External dependencies including payment gateways, digital wallet APIs, SMS/email notification services, and age verification databases are abstracted through API integration layers to allow for provider flexibility and future replacement without core system disruption.

Developers should note that the seat reservation and checkout flow is time-critical, requiring WebSocket or long-polling implementations to push real-time seat status updates to all active clients simultaneously. The concession ordering subsystem operates semi-independently and communicates with the core ticketing system only through the shared ticket identifier, allowing modular development and testing of each subsystem in isolation.

## 4.2 Architectural Design

### 4.2.1 Architectural Diagram

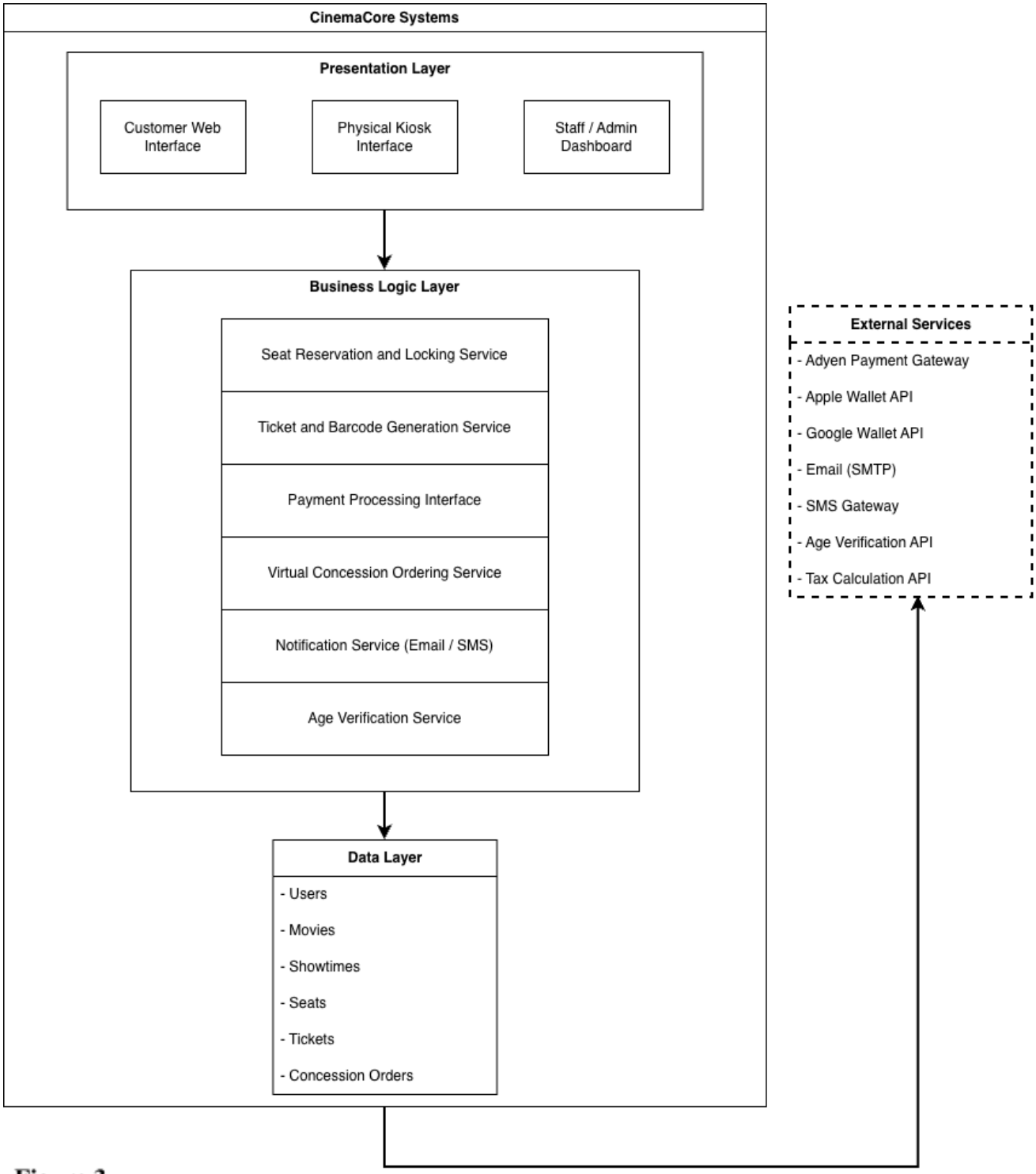


**Figure 3**

### 4.2.2 Architectural Description

The CinemaCore System employs a layered architectural design consisting of a Presentation Layer, a Business Logic Layer, and a Data Layer. This structure separates user interaction, core processing, and persistent data management to improve maintainability, scalability, and security.

The Presentation Layer includes the Customer Web Interface, Physical Kiosk Interface, and Staff/Admin Dashboard. These interfaces are responsible for user input and output and communicate with the system through secure HTTP/HTTPS connections. The Business Logic Layer contains the core system services, including seat reservation and locking, ticket and barcode generation, payment processing, virtual concession ordering, notification handling, and age verification. This layer enforces all system rules and coordinates interactions between internal components and external services.

The Data Layer provides persistent storage for users, movies, showtimes, seats, tickets, and concession orders. All database access is performed exclusively by the Business Logic Layer to ensure data integrity and access control. External services, including payment gateways, digital wallet APIs, notification services, tax calculation systems, and age verification services, are accessed through secure API integrations and operate outside the CinemaCore System boundary.

## 4.3 Software Architecture (SWA) Design

### 4.3.1 SWA Diagram

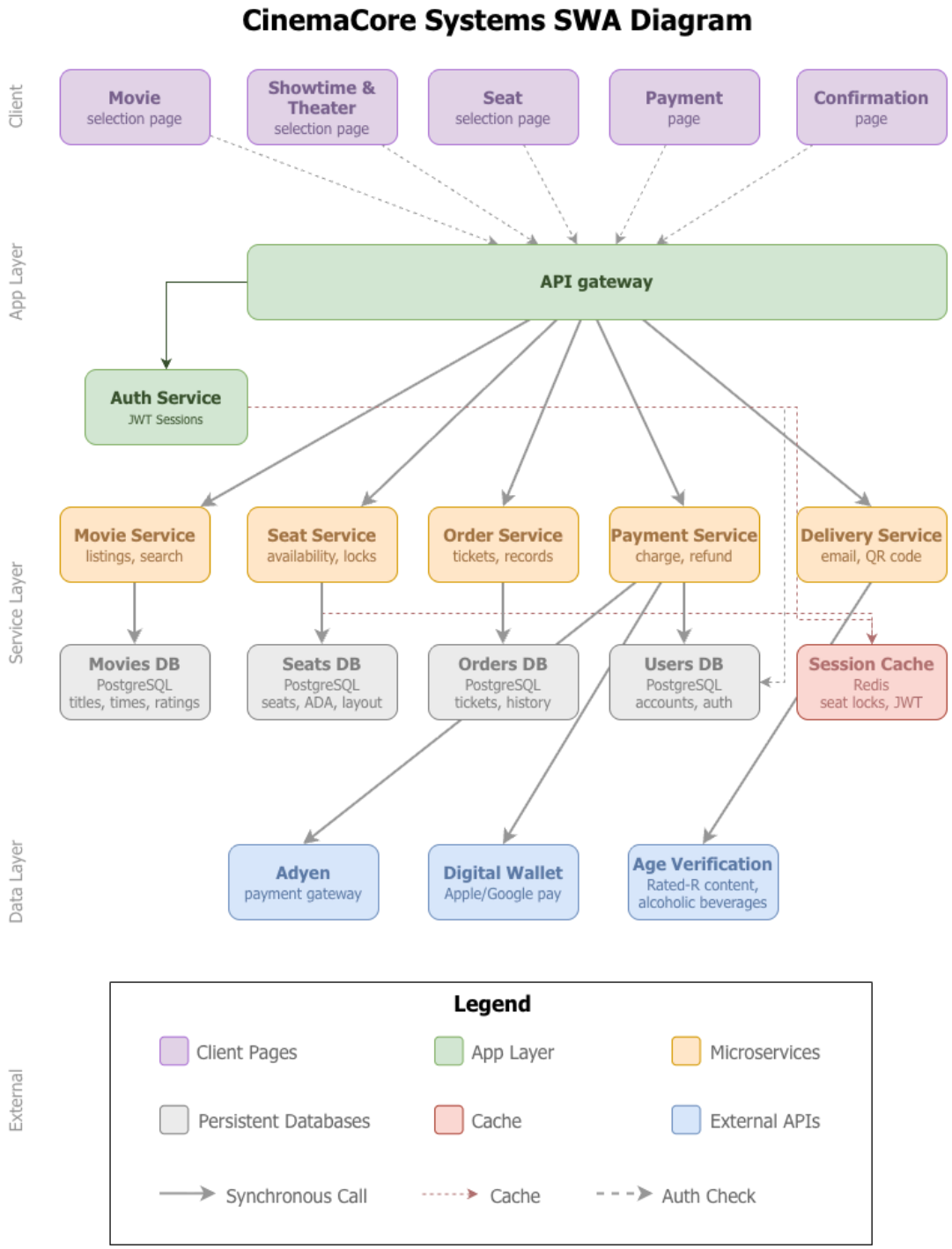


**Figure 4**

### 4.3.2 SWA Description

The CinemaCore Systems Architectural Design follows a microservices-based design centered around an API gateway, which serves as the single entry point for all client interactions. The diagram illustrates how client pages communicate with backend services through the gateway, ensuring controlled access, clear service boundaries, and separation of concerns across system layers.

The ticket purchasing process begins at the client layer, where users navigate through the Movie Selection, Showtime & Theater Selection, Seat Selection, Payment, and Confirmation pages. Each client request is routed through the API gateway, which performs request routing and authentication checks before forwarding the request to the appropriate service.

Authentication is handled by the Auth Service, which manages user sessions using JWTs. Session data is stored in a Redis cache to enable fast validation without frequent access to the Users Database. This ensures that authentication remains logically separate from other services while maintaining low-latency performance.

Each core functionality is implemented as an independent microservice. The Movie Service retrieves movie listings and metadata from the Movies Database. The Seat Service manages seat availability and enforces temporary seat locks using Redis to prevent concurrent booking conflicts. The Order Service is responsible for ticket creation and maintaining purchase records in the Tickets & Orders Database, ensuring transactional consistency.

The Payment Service handles all payment-related operations and communicates securely with the external payment provider Adyen. No payment card data is stored within the system. The Delivery Service manages ticket distribution, including QR code generation and integration with digital wallet platforms.

External services are accessed only through their corresponding microservices. Digital wallet APIs are used for ticket delivery, and age verification APIs are invoked when required for restricted content or purchases. This design isolates third-party dependencies and prevents direct client interaction with external systems.

Each service follows a service-ownership model, where it exclusively manages its own database. Direct database access across services is not permitted, as all inter-service communication occurs through APIs routed by the gateway. The Redis cache is used for time-sensitive operations such as seat locking and session management, while relational databases remain the source of truth for persistent data.

Overall, the architecture maintains the user-centered flow of the ticket purchasing process while improving scalability, fault isolation, and maintainability through centralized request routing, independent services, and clear data ownership boundaries.

### 4.3.3 SWA Revisions

The original architectural diagram exhibited several structural and clarity limitations that were addressed in the revised design. The following modifications were made:

- **Consolidation of data storage components:** The original design included two separate and redundant "Tickets & Orders" databases, which were functionally equivalent and interchangeable. In the revised architecture, these have been consolidated into a single Orders DB, which now encapsulates both ticket transaction records and historical order data, improving data consistency and reducing redundancy.
- **Introduction of a layered architectural model:** The original diagram lacked explicit architectural layering, resulting in unclear separation of concerns (e.g., login and signup components were represented as isolated UI elements with loosely defined data flows). The revised design introduces a clearly defined multi-tier architecture consisting of client layer (UI pages), application layer, service layer, and data layer, improving system modularity, maintainability, and traceability of data flow.
- **Addition of caching and session management layer:** The original architecture did not include a caching mechanism. The revised design incorporates a Redis-based cache layer, which supports session management and implements seat reservation locking to prevent race conditions and double-booking in high-concurrency scenarios.
- **Improved diagrammatic notation and documentation:** The original diagram did not provide a legend or notation guide for interpreting visual elements such as arrow types and container boundaries. The revised diagram includes a formal legend defining all connectors, arrow styles, and structural containers, thereby improving interpretability and reducing ambiguity in system communication flows.
- **Improved alignment between system flow and user interaction**: The revised SWA diagram better reflects the actual user journey from movie selection through ticket delivery, ensuring that system components directly correspond to real-world user actions and improving traceability between requirements and architecture.

## 4.4 Development Plan

The development of the CinemaCore System will be executed according to a structured and time-phased development plan. Tasks are organized to follow a logical progression from requirements validation and architectural design to implementation, testing, and deployment, and the project timeline allocates defined periods for each major development phase to ensure that functional requirements are validated before proceeding to subsequent stages.

### 4.4.1 Task Breakdown

The CinemaCore System development is divided into the following high-level tasks:

#### 4.4.1.1 Requirements Review and Finalization

This task involves performing a comprehensive review of the Software Requirements Specification (SRS) to ensure that all functional and non-functional requirements are clearly defined, technically feasible, and internally consistent before development begins. The review process includes identifying potential ambiguities, resolving requirement conflicts, validating assumptions with stakeholders, and ensuring that each requirement

can be realistically implemented within the project timeline and constraints. Additionally, this phase establishes traceability between requirements and future development tasks to support verification and testing later in the project lifecycle.
**Responsible Team:** Requirements Engineering Team

#### 4.4.1.2 System Architecture and Design

During this phase, the high-level requirements defined in the SRS are translated into a detailed technical architecture for CinemaCore Systems. This includes defining the overall system structure, identifying major software components, specifying class relationships, designing data flows, and establishing communication between modules such as ticket management, seat reservations, and concession ordering. Architectural diagrams, system models, and interface specifications are produced to guide developers during implementation and ensure the system is scalable, maintainable, and modular.
**Responsible Team:** Software Architecture and Systems Design Team

#### 4.4.1.3 User Interface Design

This task focuses on designing the visual and interactive components of the CinemaCore system that will be presented to end users. Activities include creating user interface layouts, designing navigation flows, defining input forms for ticket purchasing and seat selection, and ensuring that the interface is intuitive and easy to use. The design process also considers usability principles, accessibility standards, and consistency across different screens. Prototypes and interface mockups may be produced to validate design decisions before implementation.
**Responsible Team:** User Experience (UX) and Frontend Development Team

#### 4.4.1.4 Core System Implementation

The core implementation phase involves developing the primary application logic that fulfills the functional requirements defined in the SRS. This includes implementing the object-oriented classes and methods specified in the system design, such as user management, ticket processing, seat reservation handling, and concession order management. Developers also integrate the various modules into a cohesive system, ensuring that interactions between components function correctly and that system behavior matches the intended design.
**Responsible Team:** Backend Software Development Team

#### 4.4.1.5 Data Management

This task focuses on designing and implementing the data structures and storage mechanisms required by CinemaCore Systems. Responsibilities include organizing the system's data models, ensuring efficient storage and retrieval of information such as users, tickets, movies, and concession orders, and maintaining data consistency across system operations. The team also ensures that data handling processes support scalability, reliability, and security while meeting the system's performance requirements.
**Responsible Team:** Database Engineering and Data Management Team

#### 4.4.1.6 External Service Integration

This phase involves connecting the CinemaCore system with external services and systems through an integration layer. This may include interfacing with third-party APIs for payment processing, ticket validation, or additional services required by the theater environment. The integration layer ensures that external communications are secure, reliable, and properly handled by the system. Developers must also manage error handling, data formatting, and communication protocols between CinemaCore and external systems.
**Responsible Team:** Integration and API Development Team

#### 4.4.1.7 Testing and Validation

The testing and validation phase ensures that CinemaCore Systems meets all functional and non-functional requirements defined in the SRS. This includes performing unit testing, integration testing, and system-level testing to verify that individual components and the overall system operate correctly. The testing team also evaluates system performance, reliability, and usability, identifying and documenting any defects or inconsistencies. Validation activities ensure that the final system behaves as expected and satisfies stakeholder requirements before deployment.
**Responsible Team:** Quality Assurance (QA) and Software Testing Team

#### 4.4.1.8 Deployment and Documentation

This final phase involves preparing CinemaCore Systems for release in a live theater environment. Tasks include configuring the deployment environment, packaging the application for installation, and ensuring that all required system components are operational. In addition, comprehensive documentation is produced to support system administrators, developers, and end users. This documentation may include installation guides, user manuals, system maintenance instructions, and technical reference materials.
**Responsible Team:** DevOps and Technical Documentation Team

### 4.4.2 Project Timeline

| Project Timeline | | | | | |
|---|---|---|---|---|---|
| **Task ID** | **Task Name** | **Start Week** | **End Week** | **Duration** | **Dependencies** |
| T1 | Requirements Review and Finalization | Week 1 | Week 1 | 1 Week | N/A |
| T2 | System Architecture and Design | Week 2 | Week 2 | 1 Week | T1 |
| T3 | User Interface Design | Week 3 | Week 3 | 1 Week | T2 |
| T4 | Implementation | Week 4 | Week 6 | 3 Weeks | T2, T3 |
| T5 | Data Management | Week 4 | Week 5 | 2 Weeks | T2 |
| T6 | External Service Integration | Week 5 | Week 6 | 2 Weeks | T4, T5 |

| Project Timeline | | | | | |
|---|---|---|---|---|---|
| T7 | Testing and Validation | Week 7 | Week 7 | 1 Week | T4, T6 |
| T8 | Deployment and Documentation | Week 8 | Week 8 | 1 Week | T7 |

## 4.5 Test Plan

This section defines the verification and validation strategy for CinemaCore Systems, mapping each test case to specific class functions, and functional and non-functional requirements established in Sections 3.2, 3.4, and 3.5 respectively. Tests are organized by type: Functional, Unit, and System. The tests target the three core subsystems: Seat Reservation, Ticket Purchase/Delivery, and Concession Ordering.

### 4.5.1 Test Scope and Coverage Strategy

The test plan targets the three core subsystems identified in the SWA Diagram (Section 4.3.1): Seat Reservation (FR-1), Ticket Purchase and Delivery (FR-2), and Virtual Concession Ordering (FR-3). Additionally, tests validate critical inverse requirements defined in Section 3.6. The three test types serve distinct verification purposes:

- **Functional Tests (TC-001 through TC-006)**: Validate that each core user-facing feature produces the correct output for given inputs, directly exercising the processing logic described in Sections 3.2.1 through 3.2.3, as well as inverse requirements defined in 3.6.
- **Unit Tests (TC-007, TC-008)**: Isolate and verify individual class methods from the UML diagram (Section 3.4), specifically Seat.lockSeat() and Showtime.updateAvailability(), which underpin concurrency safety and real-time seat map accuracy.
- **System / End-to-End Tests (TC-009, TC-010):** Validate complete user workflows that cross multiple subsystems, ensuring that component integrations, such as concession linkage to ticket identifiers and user account authentication, function correctly as a whole.

### 4.5.2 Test Priority

Tests are assigned priority P0 (critical) or P1 (important). All P0 tests must pass before the system proceeds to deployment (Task T8). P1 tests must pass before final release but are not blockers for initial deployment readiness. P0 tests cover seat reservation integrity, payment processing, ticket generation and delivery, ticket validation, user account management, and core unit method correctness. P1 tests cover dynamic pricing display and end-to-end concession ordering.

### 4.5.3 Test Case Specification

This section provides detailed specifications for each test defined in the test plan. Each test description includes the features under test, the scope of coverage, and the associated test sets or test vectors utilized during execution.

#### 4.5.3.1 TC-001 Real-time Seat Availability Display

- **Features being tested:** This test targets the real-time seat map rendering functionality specified in FR-1. Specifically, it verifies that when a user navigates

to the seat selection page, the system correctly retrieves seat status from the database and visually distinguishes between reserved, temporarily locked, and available seats.

- **Test Set / Vectors:** The precondition is that at least one movie showtime exists in the system with seats in all three states: available, locked by an active session, and reserved by a user. The input sequence is: (1) select a movie title, (2) select a showtime, (3) display the seat selection page.
- **Coverage:** This test covers the real-time database query and front-end rendering pipeline from the Theaters/Seats Database through the Select Seat Page (Section 4.3.1). A correct pass result confirms that the seat map accurately reflects the database state, which is the foundational requirement for preventing double-booking. If this test fails, all downstream seat reservation and checkout functionality is invalid. The test plan also considers failure scenarios such as payment failures, seat lock expiration, and duplicate ticket validation to ensure system reliability under real-world conditions.

#### 4.5.3.2 TC-002 Dynamic Tiered Pricing by Seat Location

- **Features being tested:** This test targets the tiered pricing logic specified in FR-1 3.2.1.3, which requires the system to apply a discount for front-row seats and a surcharge for premium center-section seats. The pricing logic executes during seat selection before the user reaches the checkout flow.
- **Test Set / Vectors:** Two distinct input vectors are exercised. Vector 1: the user selects a seat in the front row of the theater map. Vector 2: the user selects a seat in the center premium section. In both cases the system must display a price summary that differs from the base ticket price, reduced for the front row and elevated for the premium section.
- **Coverage:** This test exercises the tiered pricing calculation within the Business Logic Layer and verifies that the price summary output rendered to the user accurately reflects the seat location surcharge and discount rules. It covers the processing clause of FR-1 3.2.1.3 and the corresponding output clause in 3.2.1.4, confirming that the breakdown of base price and location-based adjustments is surfaced correctly to the customer before purchase.

#### 4.5.3.3 TC-003 Failed Payment Releases Locked Seats

- **Features being tested:** This test targets the error handling clause of FR-2, which requires that if payment authorization fails, the system must cancel the transaction, release the locked seats back to available, and display an appropriate error message. It directly validates one of the failure modes in the system.
- **Test Set / Vectors:** The precondition is that one or more seats are in a temporarily locked state associated with the current session. The input is deliberately invalid, a card number that will be declined by the payment gateway. The expected outcome is a three-part result: the transaction is cancelled, the seats return to an available state visible to other users, and an error message is displayed to the user.

- **Coverage:** This test covers the payment processing subsystem's integration with the seat locking mechanism. It confirms that a failed external call to Adyen correctly triggers the seat release path in the Business Logic Layer, preventing seats from being held indefinitely after a failed purchase, an inverse requirement explicitly stated in 3.6.1.

#### 4.5.3.4 TC-004 Unique Ticket ID and Barcode Generation

- **Features being tested:** This test targets the ticket generation subsystem specified in FR-2 3.2.2.3, which requires the system to generate a unique ticket identification number and associated barcode upon successful payment. Uniqueness is a hard requirement, duplicate ticket IDs are explicitly prohibited by inverse requirement 3.6.1.
- **Test Set / Vectors:** The precondition is a completed and authorized payment. The input is a successful end-to-end purchase transaction. The expected output is a ticket record containing a unique ID and a scannable barcode that is linked to the specific screening room, showtime, and seat number selected by the user.
- **Coverage:** This test exercises the ticket generation subsystem within the Business Logic Layer and verifies the output stored in the Tickets and Orders Database. It confirms that the Ticket class constructor (Section 3.4.2.2) correctly instantiates a ticket with a non-duplicate identifier and that the barcode is scannable, covering the non-duplication constraint in 3.6.1 and the linkage requirement in 3.2.2.3.

#### 4.5.3.5 TC-005 Multi-Channel Ticket Delivery via Email and SMS

- **Features being tested:** This test targets the ticket delivery subsystem specified in FR-2 3.2.2.4, which requires that digital tickets be sent via email, SMS, and/or digital wallet. The test specifically validates the email and SMS delivery channels to confirm that the ticket reaches the customer in a scannable format through both paths.
- **Test Set / Vectors:** Two delivery vectors are exercised within a single purchase flow. Vector 1: the user selects email delivery and provides a valid email address, the system must transmit a digital ticket to that address. Vector 2: the user selects SMS delivery and provides a valid phone number, the system must transmit a ticket link via text message. In both cases the delivered ticket must be scannable at a theater entry point.
- **Coverage:** This test covers the Notification and Delivery subsystem's integration with the external Email/SMS service shown in the SWA Diagram (Section 4.3.1). It validates that the ticket delivery component correctly interfaces with the SMTP and SMS gateway services and that the inverse requirement of 3.6.5 is satisfied, the ticket is not marked as successfully delivered unless a confirmed transmission has occurred.

#### 4.5.3.6 TC-006 Duplicate Ticket Scan Rejection

- **Features being tested:** This test targets the ticket validation subsystem and the inverse requirement stated in 3.6.6: the system shall not allow the same ticket to

be scanned more than once for theater entry. It also validates the reliability requirement in 3.5.2, which states that the system must reliably prevent double-booking and unauthorized re-entry under all conditions.

- **Test Set / Vectors:** The precondition is a valid, previously unused ticket with a generated barcode. The test executes two sequential scan attempts using the same barcode. Vector 1: the first scan, this should succeed and grant entry. Vector 2: the second scan of the identical barcode immediately after, this must be rejected by the system as invalid.
- **Coverage:** This test covers the Ticket Validation subsystem's stateful tracking of scan history. It confirms that once a ticket barcode has been successfully read and entry granted, the system marks that ticket as used and blocks all subsequent scan attempts. This is the primary safeguard against ticket sharing and unauthorized duplicate entry, directly exercising the constraint in 3.6.6.

#### 4.5.3.7 TC-007 Seat.lockSeat() Prevents Concurrent Selection

- **Features being tested:** This test targets the lockSeat() method of the Seat class (Section 3.4.2.3), which is responsible for preventing double-booking by marking a seat as unavailable to other users as soon as it is selected.
- **Test Set / Vectors:** The test simulates two concurrent user sessions. Vector 1: User A selects a seat, triggering lockSeat() on that Seat object, the system should set isLocked to true. Vector 2: immediately after, User B attempts to select the same seat. The expected result is that the seat is returned as locked and User B's selection attempt is blocked. The isLocked() method is then called to confirm the boolean state is correctly set to true.
- **Coverage:** This unit test directly exercises the lockSeat() and isLocked() methods of the Seat class in isolation. It confirms that the seat locking mechanism functions correctly at the object level before integration with the broader seat map rendering and database synchronization layers. This test covers the reliability failure mode in the system, or concurrent seat selection leading to double-booking, as required by FR-1 3.2.1.3 and the reliability constraint in 3.5.2.

#### 4.5.3.8 TC-008 Showtime.updateAvailability() Reflects State Changes

- **Features being tested:** This test targets the updateAvailability() method of the Showtime class (Section 3.4.2.5), which is responsible for updating the availability status of a showing as seats are locked, purchased, or released. It verifies that the method correctly transitions state and that those transitions are accurately reflected in the seat map.
- **Test Set / Vectors:** Three sequential state transitions are exercised. Vector 1: a seat is locked, updateAvailability() is called and the seat map is refreshed, expected to show the seat as locked. Vector 2: the purchase is completed, updateAvailability() is called again, expected to show the seat as permanently reserved. Vector 3: a separate seat hold expires, updateAvailability() is called, expected to return that seat to available. After each vector, getAvailability() is called to assert the correct boolean state.

- **Coverage:** This test covers the updateAvailability() and getAvailability() methods of the Showtime class across all three relevant state transitions. It confirms that the method correctly handles lock, purchase, and release events, which is the foundation of the real-time seat map accuracy required by FR-1 and the performance requirement in 3.5.1 that seat availability must update in real time across all users.

#### 4.5.3.9 TC-009 End-to-end Concession Order Flow

- **Features being tested:** This test targets the full concession ordering pipeline specified in FR-3, from item selection through payment, order linkage to a seat, transmission to the staff interface, and real-time status updates. It is the most comprehensive integration test for the concession subsystem and validates that all components, the Virtual Concession subsystem, Order Management subsystem, and Staff Operations Interface, operate correctly together.
- **Test Set / Vectors:** The precondition is an existing valid ticket with a confirmed seat assignment and at least one concession item marked as available in the menu. The input sequence is: (1) browse the virtual concession menu and select items, (2) submit the order with the associated ticket identifier, (3) observe the staff dashboard for the incoming order. The expected outcome is that the order appears on the staff interface correctly linked to the seat and screening room, and that the order status progresses through the defined states: Received, In Preparation, and Out for Delivery.
- **Coverage:** This test exercises the full FR-3 processing pipeline: menu display, order linkage to the ticket identifier per 3.2.3.3, food transmission to the staff interface, and real-time status updates per 3.2.3.4. It validates that the concession subsystem correctly communicates with the ticketing system through the shared ticket identifier and that the staff operations interface accurately receives and displays order data, confirming the end-to-end integration required before the system can go live.

#### 4.5.3.9.1 TC-010 New User Account Creation and Login

- **Features being tested:** This test targets the user account management functionality described in Section 3.2, specifically the ability for a new customer to create an account, authenticate with those credentials, and successfully access the movie browsing and account features of the system. It validates the User class instantiation (Section 3.4.2.1) and the interaction between the Login/Sign Up Page and the User Accounts Database shown in the SWA Diagram (Section 4.3.1).
- **Test Set / Vectors:** The precondition is that no existing account is associated with the test email address. The input sequence is: (1) navigate to the create account page and submit a new username, email, and password, (2) log in using the newly created credentials, (3) navigate to the account page to confirm access to user-specific features such as loyalty points and ticket history. The expected

outcome is that the account is created, login succeeds, and the account page is accessible with the correct user profile data.

- **Coverage:** This test covers the full authentication flow from front-end form submission through the User Accounts Database and back to the authenticated session state. It confirms that the User class is correctly instantiated with all required attributes (id, name, email, phone, type, points) and that the system enforces clean separation between authentication and seat selection as described in the SWA architecture. A passing result confirms the system supports the account-based ticket retrieval method described in 3.5.4.

### 4.5.4 Test Plan Table

| Test Case Table - CinemaCore Systems | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| **TestCaseId** | **Type** | **Component** | **Priority** | **Description/Test Summary** | **Pre-requisites** | **Test Steps** | **Expected Result** | **Actual Result** | **Status** | **Test Executed By** |
| TC-001 | Functional | Seat Selection | P0 | Verify the system displays real-time seat availability | Viewing a movie showtime | 1. Select a movie and showtime<br>2. Open seat selection page | Seat map should display reserved, locked, and available seats correctly | Seat map displays reserved, locked, and available seats | Pass | User_Exp_Tester |
| TC-002 | Functional | Pricing | P1 | Verify dynamic pricing based on seat location | Seat map visible | 1. Select front row seat<br>2. Select center seat | Different price tiers displayed | Front row price was discounted, while center seat had premium pricing | Pass | User_Exp_Tester |
| TC-003 | Functional | Payment | P0 | Verify failed payment and release seats | Seat locked | 1. Enter invalid card<br>2. Submit | Transaction fails and seats released | Payment declined and seat returned to available | Pass | Payment_Tester |
| TC-004 | Functional | Ticket Generation | P0 | Verify unique ticket and ID creation | Payment completed | 1. Complete purchase | Unique ticket ID generated | User was given a unique ticket ID and barcode | Pass | Payment_Tester |

| | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| TC-005 | Functional | Ticket Delivery | P0 | Verify ticket can be delivered via email and SMS | Ticket purchased | 1. Deliver ticket through email<br>2. Deliver ticket thorugh SMS | Scannable ticket sent to email and SMS texts | User received ticket through email and SMS | Pass | User_Exp_Tester |
| TC-006 | Functional | Ticket Validation | P0 | Verify ticket can not be scanned twice | Virtual ticket unique ID is generated and scannable | 1. Scan ticket barcode<br>2. Scan barcode again | Barcode during second attempt is unscannable | Barcode during second attempt was unscannable | Pass | User_Exp_Tester |
| TC-007 | Unit | LockSeat | P0 | Lock seat when it has already been selected by another user | Viewing seat availability | 1. Seat selected by another user<br>2. Try to select seat that was just selected | Selected seat is locked from other users | Correct seat is locked and cannot be selected by other users | Pass | Unit_Tester_01 |
| TC-008 | Unit | UpdateAvailability | P0 | Verify that the UpdateAvailability works correctly, updating seat status after a seat is locked, purchased, or released. | A valid movie and showtime exist<br>A seat is available<br>Seat map is loaded | 1. Lock seat<br>2. Complete purchase or release seat<br>3. Refresh seat map | Seat status updates correctly after each action | Seat map reflects correct availability | Pass | Unit_Tester_02 |
| TC-009 | System | End to End Concession Purchase | P1 | Verify full concession order flow works correctly. | Valid ticket, items available | 1. Select items<br>2. Submit order<br>3. Check staff dashboard | Order is processed, linked to seat, and sent to staff | Order completed and visible to staff | Pass | System_Tester_01 |
| TC-010 | System | Create Login | P0 | Verify new user can create an account login and use it to view movies | New user selects create account on website | 1. Create account<br>2. Log in<br>3. Access account page | Account created and login successful | User logged in and accessed account features | Pass | System_Tester_02 |

# 5. Data Management Strategy

This section defines how data is stored, accessed, and protected across the system. The design prioritizes data integrity, scalability, and fault isolation by partitioning data into domain-specific databases and using a caching layer for time-sensitive operations. A service-ownership model is adopted, where each microservice exclusively manages its own data store, ensuring clear boundaries and reducing inter-service coupling. The system leverages relational databases for structured, transactional data and an in-memory cache for low-latency operations.

## 5.1 Technology Choice

The system uses four PostgreSQL relational databases and one Redis cache, each with a distinct responsibility:

- Users DB - stores account credentials (hashed passwords), profile information, and payment method references.
- Movies & Showtimes DB - stores titles, ratings, showtimes, and genre metadata. Read-heavy, rarely mutated after initialization.
- Theaters & Seats DB - stores theater layouts, seat maps, ADA designations, and showing schedules. Complex relational structure (a theater has many showings, each showing has many seats) makes relational SQL the natural fit.
- Tickets & Orders DB - stores confirmed ticket purchases and order history. This is the most sensitive table, as it must support ACID transactions to ensure a payment and a ticket issuance are never partially applied.
- Redis cache - handles two time-critical functions: (1) seat reservation locks (FR-1), and (2) JWT session tokens for fast auth validation without hitting the Users DB on every request.

Each service owns its own database, following a service-ownership pattern. This means the Movie service cannot directly query the Tickets & Orders DB, it must go through the Ticket & Order service's API. This isolation provides three benefits: it prevents a schema change in one domain from breaking another, it allows each database to be scaled independently (e.g. the Theaters & Seats DB under heavy seat-check load can scale without touching the Tickets & Orders DB), and it makes the boundaries of each service explicit and auditable.

This architecture also supports horizontal scalability. Each database and service can be scaled independently based on demand. For example, the Theaters & Seats database can scale to handle high-frequency seat availability during peak times without ruining the performance.

## 5.2 Database Organization

The system splits data across four logical databases, each aligned to a functional domain in the SWA (Section 4.3.1). This mirrors the modular architecture described in Section 4.1: a failure in the concession domain does not affect ticket validation, and each subsystem can be scaled and maintained independently.

### 5.2.1 Users

This database manages all user-related data, including authentication credentials, profile information, and loyalty program details. It is optimized for secure storage and frequent read/write operations during login, account updates, and purchase flows. Sensitive data such as passwords is never stored in plaintext and is protected using strong hashing mechanisms.

| users | | |
|---|---|---|
| **Column** | **Type** | **Notes** |
| **user_id (PK)** | SERIAL | Unique identifier |
| name, email, phone | VARCHAR | Contact and account info |
| password_hash | VARCHAR(255) | Bcrypt-hashed, plaintext never stored |

| | | |
|---|---|---|
| age | INTEGER | Used for age verification (R-rated/alcohol) |
| account_type, points | VARCHAR/INTEGER | Loyalty program data |

### 5.2.2 Movies & Showtimes

This database stores relatively static content related to movies and their scheduled showtimes. It is primarily read-heavy, with infrequent updates when new movies or schedules are introduced. The schema is designed to efficiently support browsing, filtering, and validation of showtime availability during ticket purchase.

| movies / showtimes | | |
|---|---|---|
| **Column** | **Type** | **Notes** |
| **movie_id (PK)** | SERIAL | Unique movie identifier |
| title, description, duration_min | VARCHAR/INTEGER | Film metadata |
| mpaa_rating | VARCHAR(10) | Enforces age-restricted purchase rules |
| **showtime_id (PK)** | SERIAL | Unique showtime identifier |
| **movie_id (FK)** | INTEGER | Links showtime to movie |
| show_date, start_time, screen_room | DATE/TIME/INT | Scheduling data |
| is_available | BOOLEAN | Controls whether tickets can be purchased |

### 5.2.3 Theaters & Seats

This database models the physical structure of theaters and the state of individual seats for each showtime. It supports complex relationships between theaters, showings, and seats, and is optimized for high-frequency queries during seat selection. It also maintains temporary and permanent seat states (e.g., locked, reserved) to support concurrency control during the booking process.

| seats | | |
|---|---|---|
| **Column** | **Type** | **Notes** |
| **seat_id (PK)** | SERIAL | Unique seat identifier |
| **showtime_id (FK)** | INTEGER | Links seat to a specific screening |
| seat_number, aisle | INTEGER/VARCHAR | Physical location |
| price_tier, base_price | VARCHAR/DECIMAL | Supports tiered pricing (FR-1) |
| is_locked, lock_expires_at | BOOLEAN/TIMESTAMP | 10-minute hold logic |
| **is_reserved, reserved_by (FK)** | BOOLEAN/INTEGER | Permanent reservation after purchase |

### 5.2.4 Tickets & Orders

This database handles all finalized transactions, including ticket issuance and concession orders. It requires strict transactional guarantees to ensure consistency between payment processing and

ticket generation. The schema enforces uniqueness (e.g., barcodes) and maintains order lifecycle states to support tracking and fulfillment.

| tickets / concession_orders / order_items | | |
|---|---|---|
| **Column** | **Type** | **Notes** |
| **ticket_id (PK)** | SERIAL | Unique ticket identifier |
| **user_id, seat_id, showtime_id (FK)** | INTEGER | Links ticket to purchase context |
| barcode | VARCHAR(64) | Unique scannable string, no duplicates allowed |
| is_scanned | BOOLEAN | Set TRUE on first scan; never reset |
| **order_id (PK)** | SERIAL | Unique concession order identifier |
| **ticket_id (FK)** | INTEGER | Order cannot exist without valid ticket |
| status | VARCHAR | Received → In Preparation → Out for Delivery → Delivered |
| **item_id (PK)** | SERIAL | Individual line item in an order |
| requires_age_verify | BOOLEAN | TRUE for alcohol items |

## 5.3 Alternatives & Tradeoffs

This section evaluates alternative data storage approaches and justifies the selected architecture based on system requirements.

| Approach | Example | Advantages | Disadvantages |
|---|---|---|---|
| **Single SQL DB (monolith)** | PostgreSQL, single instance | Strong referential integrity via native constraints, simpler deployment and querying, easier to maintain consistency. | Limited scalability, tight coupling between domains, higher risk of system-wide impact from schema changes or failures. |
| **Multi-DB SQL (chosen)** | PostgreSQL, multiple instances | Clear service boundaries, independent scaling per domain, improved fault isolation, aligns with microservices architecture. | No native cross-database FK's, increased operational complexity, requires application-level validation for cross-domain relationships. |
| **Document Store** | MongoDB | Flexible schema design, horizontal scalability, well-suited for unstructured or rapidly evolving data. | Weaker support for complex joins and transactions, less natural fit for relational data (e.g., seat-to-ticket relationships), enforcing constraints (uniqueness, consistency) is more difficult. |
| **SQL + Redis cache (recommended)** | PostgreSQL + Redis | Combines durable, consistent storage with low-latency access, ideal for time-sensitive operations (e.g., seat locking, session validation), reduces database load. | Increased system complexity, requires cache invalidation strategies, introduces potential consistency challenges between cache and source of truth. |

The selected multi-database SQL approach provides the best balance between consistency, scalability, and modularity. While it sacrifices native cross-database referential integrity, this is mitigated through strict validation in the Business Logic Layer and well-defined service APIs. This tradeoff is acceptable given the need for independent scaling and fault isolation across subsystems.

## 5.4 Security Considerations

This section outlines the measures taken to protect sensitive data and ensure secure system operation. The system follows industry best practices for data protection, encryption, and access control. Measures include:

- No payment card data is stored in any of the system's databases. All payment processing is handled by Adyen (PCI-DSS compliant).
- Passwords are stored as Bcrypt hashes only (work factor ≥ 12).
- Database connections use TLS in transit, volumes are encrypted at rest.
- Staff accounts are restricted to reading and updating concession order status, they cannot access user credentials or payment records.